Implementation of stack using arrays :

Programiz C Online Compiler

Programiz PRO ›

main.c

Share Run

```c
#include <stdio.h>
#define MAX 5

int stack[MAX];
int top = -1;

void push(int x) {
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
    } else {
        stack[++top] = x;
        printf("%d pushed\n", x);
    }
}

void pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
    } else {
        printf("%d popped\n", stack[top--]);
    }
}

```

Output

Clear

```
10 pushed
20 pushed
30 pushed
Stack elements: 10 20 30
30 popped
Stack elements: 10 20


=== Code Execution Successful ===
```

main.c

Share Run

```c
}

void displayStack() {
    if (top == -1) {
        printf("Stack is empty\n");
    } else {
        printf("Stack elements: ");
        for (int i = 0; i <= top; i++) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}

int main() {
    push(10);
    push(20);
    push(30);
    displayStack();
    pop();
    displayStack();
    return 0;
}
```

Output

Clear

```
10 pushed
20 pushed
30 pushed
Stack elements: 10 20 30
30 popped
Stack elements: 10 20


=== Code Execution Successful ===
```

Implementation of stack using Linked list:

Programiz C Online Compiler

Programiz PRO ›

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Stack structure
struct Stack {
    struct Node* top;
};

// Initialize stack
void init(struct Stack* s) {
    s->top = NULL;
}

// Check if empty
int empty(struct Stack* s) {
    return (s->top == NULL);
}
```

Output

Clear

```
10 pushed to stack
20 pushed to stack
30 pushed to stack
Top element: 30
Popped: 30
Popped: 20
Is stack empty? No

=== Code Execution Successful ===
```

Programiz C Online Compiler

Programiz PRO ›

main.c

Share Run

```c
// Push operation
void push(struct Stack* s, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node
        ));

    newNode->data = data;
    newNode->next = s->top;
    s->top = newNode;

    printf("%d pushed to stack\n", data);
}

// Pop operation
int pop(struct Stack* s) {
    if (empty(s)) {
        printf("Stack Underflow\n");
        return -1;  // acts like None
    }

    struct Node* temp = s->top;
    int popped = temp->data;
```

Output

Clear

```
10 pushed to stack
20 pushed to stack
30 pushed to stack
Top element: 30
Popped: 30
Popped: 20
Is stack empty? No


=== Code Execution Successful ===
```

Programiz C Online Compiler

Programiz PRO ›

main.c

Share Run

```c
    }

    struct Node* temp = s->top;
    int popped = temp->data;

    s->top = temp->next;
    free(temp);

    return popped;
}

// Peek operation
int peek(struct Stack* s) {
    if (empty(s)) {
        printf("Stack is empty\n");
        return -1;
    }

    return s->top->data;
}

// Main function (example usage)
int main() {
```

Output

Clear

```
10 pushed to stack
20 pushed to stack
30 pushed to stack
Top element: 30
Popped: 30
Popped: 20
Is stack empty? No


=== Code Execution Successful ===
```

Programiz C Online Compiler

Programiz PRO ›

main.c

Share Run

```c
    }

    return s->top->data;
}

// Main function (example usage)
int main() {
    struct Stack s;
    init(&s);

    push(&s, 10);
    push(&s, 20);
    push(&s, 30);

    printf("Top element: %d\n", peek(&s));

    printf("Popped: %d\n", pop(&s));
    printf("Popped: %d\n", pop(&s));

    printf("Is stack empty? %s\n", empty(&s) ? "Yes" : "No");

    return 0;
}
```

Output

Clear

```
10 pushed to stack
20 pushed to stack
30 pushed to stack
Top element: 30
Popped: 30
Popped: 20
Is stack empty? No


=== Code Execution Successful ===
```

Queue implementation using arrays:

Programiz C Online Compiler

Programiz PRO ›

main.c

Share Run

```c
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
    } else {
        if (front == -1) front = 0;
        queue[++rear] = x;
        printf("%d enqueued\n", x);
    }
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
    } else {
        printf("%d dequeued\n", queue[front++]);
    }
}
```

Output

Clear

```
10 enqueued
20 enqueued
30 enqueued
Queue elements: 10 20 30
10 dequeued
Queue elements: 20 30


=== Code Execution Successful ===
```

Programiz C Online Compiler

Programiz PRO ›

main.c

Share Run

```c
}

void displayQueue() {
    if (front == -1 || front > rear) {
        printf("Queue is empty\n");
    } else {
        printf("Queue elements: ");
        for (int i = front; i <= rear; i++) {
            printf("%d ", queue[i]);
        }
        printf("\n");
    }
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    displayQueue();
    dequeue();
    displayQueue();
    return 0;
}
```

Output

Clear

```
10 enqueued
20 enqueued
30 enqueued
Queue elements: 10 20 30
10 dequeued
Queue elements: 20 30


=== Code Execution Successful ===
```

Queue implementation using Linked lists:

Programiz C Online Compiler — Programiz PRO ›

main.c — Share — Run

```c
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Queue structure
struct Queue {
    struct Node *front, *rear;
};

// Initialize queue
void init(struct Queue* q) {
    q->front = q->rear = NULL;
}

// Check if empty
int is_empty(struct Queue* q) {
    return (q->front == NULL);
}
```

Output — Clear

```
Front element: 10
Is queue empty? No


=== Code Execution Successful ===
```

Programiz C Online Compiler — Programiz PRO ›

main.c — Share — Run

```c
// Enqueue operation
void enqueue(struct Queue* q, int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node
        ));
    newNode->data = data;
    newNode->next = NULL;

    if (q->rear == NULL) {
        q->front = q->rear = newNode;
        return;
    }

    q->rear->next = newNode;
    q->rear = newNode;
}

// Front operation
int front(struct Queue* q) {
    if (is_empty(q)) {
        printf("Queue is empty\n");
        return -1; // acts like None
    }
    return q->front->data;
```

Output — Clear

```
Front element: 10
Is queue empty? No


=== Code Execution Successful ===
```

Programiz C Online Compiler — Programiz PRO ›

main.c — Share — Run

```c
// Front operation
int front(struct Queue* q) {
    if (is_empty(q)) {
        printf("Queue is empty\n");
        return -1; // acts like None
    }
    return q->front->data;
}

// Example usage
int main() {
    struct Queue q;
    init(&q);

    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);

    printf("Front element: %d\n", front(&q));
    printf("Is queue empty? %s\n", is_empty(&q) ? "Yes" : "No");

    return 0;
}
```

Output — Clear

```
Front element: 10
Is queue empty? No


=== Code Execution Successful ===
```

Implementatin of Binary tree :

Programiz C Online Compiler

Programiz PRO ›

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

// TreeNode structure
struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};

// Create new node
struct TreeNode* createNode(int data) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof
        (struct TreeNode));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Insert (simple level-order insertion)
struct TreeNode* insert(struct TreeNode* root, int data) {
    if (root == NULL) {
        return createNode(data);
```

Output

Clear

```
Pre-order traversal: 10 20 40 50 30

=== Code Execution Successful ===
```

main.c

Share Run

```c
    // Use queue for level-order insertion
    struct TreeNode* queue[100];
    int front = 0, rear = 0;

    queue[rear++] = root;

    while (front < rear) {
        struct TreeNode* temp = queue[front++];

        if (temp->left == NULL) {
            temp->left = createNode(data);
            return root;
        } else {
            queue[rear++] = temp->left;
        }

        if (temp->right == NULL) {
            temp->right = createNode(data);
            return root;
        } else {
            queue[rear++] = temp->right;
        }
    }
```

Output

Clear

```
Pre-order traversal: 10 20 40 50 30

=== Code Execution Successful ===
```

main.c

Share Run

```c
void pre_order_traversal(struct TreeNode* root) {
    if (root == NULL)
        return;

    printf("%d ", root->data);          // Visit root
    pre_order_traversal(root->left);    // Left
    pre_order_traversal(root->right);   // Right
}

// Main function
int main() {
    struct TreeNode* root = NULL;

    root = insert(root, 10);
    insert(root, 20);
    insert(root, 30);
    insert(root, 40);
    insert(root, 50);

    printf("Pre-order traversal: ");
    pre_order_traversal(root);

    return 0;
```

Output

Clear

```
Pre-order traversal: 10 20 40 50 30

=== Code Execution Successful ===
```

Binary Search Tree (BST) Implementation :

main.c

```c
#include <stdio.h>
#include <stdlib.h>

// TreeNode structure
struct TreeNode {
    int data;
    struct TreeNode* left;
    struct TreeNode* right;
};

// Create new node
struct TreeNode* createNode(int data) {
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof
        (struct TreeNode));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

// Insert into BST
struct TreeNode* insert(struct TreeNode* root, int key) {
    if (root == NULL)
        return createNode(key);
```

Output

```
Inorder Traversal: 20 30 40 50 60 70 80
Postorder Traversal: 20 40 30 60 80 70 50

=== Code Execution Successful ===
```

main.c

```c
    if (key < root->data)
        root->left = insert(root->left, key);
    else if (key > root->data)
        root->right = insert(root->right, key);

    return root;
}

// Inorder Traversal (Left → Root → Right)
void in_order_traversal(struct TreeNode* root) {
    if (root == NULL)
        return;

    in_order_traversal(root->left);
    printf("%d ", root->data);
    in_order_traversal(root->right);
}

// Postorder Traversal (Left → Right → Root)
void post_order_traversal(struct TreeNode* root) {
    if (root == NULL)
        return;

```

Output

```
Inorder Traversal: 20 30 40 50 60 70 80
Postorder Traversal: 20 40 30 60 80 70 50

=== Code Execution Successful ===
```

Programiz C Online Compiler

main.c

```c
    printf("%d ", root->data);
}

// Main function
int main() {
    struct TreeNode* root = NULL;

    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);

    printf("Inorder Traversal: ");
    in_order_traversal(root);

    printf("\nPostorder Traversal: ");
    post_order_traversal(root);

    return 0;
}
```

Output

```
Inorder Traversal: 20 30 40 50 60 70 80
Postorder Traversal: 20 40 30 60 80 70 50

=== Code Execution Successful ===
```